LunarLander DQN Project Design

1.1 Problem Statement

- DQN is hard to understand from equations alone, so I want to see how it learns a control policy and whether it can improve over a random LunarLander agent.

1.2 Proposed Solution

- A Python demo that recreates the main DQN ideas from Mnih et al. (2015) for Gymnasium's LunarLander-v3, including a Q-network, experience replay, a target network, a training loop, and evaluation against my random baseline.

2.1 Eisenhower Matrix

	Urgent	Not Urgent
Important	Do Implement PyTorch Q-network Add replay memory Train and evaluate against baseline (baseline lrdy made)	Schedule Plot reward and loss Experiment with DQN or hyperparameters Export, polish
Not Important	Delegate Gymnasium managing LunarLander env Box2d manages physics PyTorch manages tensors and gradients	Delete Custom physics sim Comparing multiple RL algorithms Creating a website using NPUS

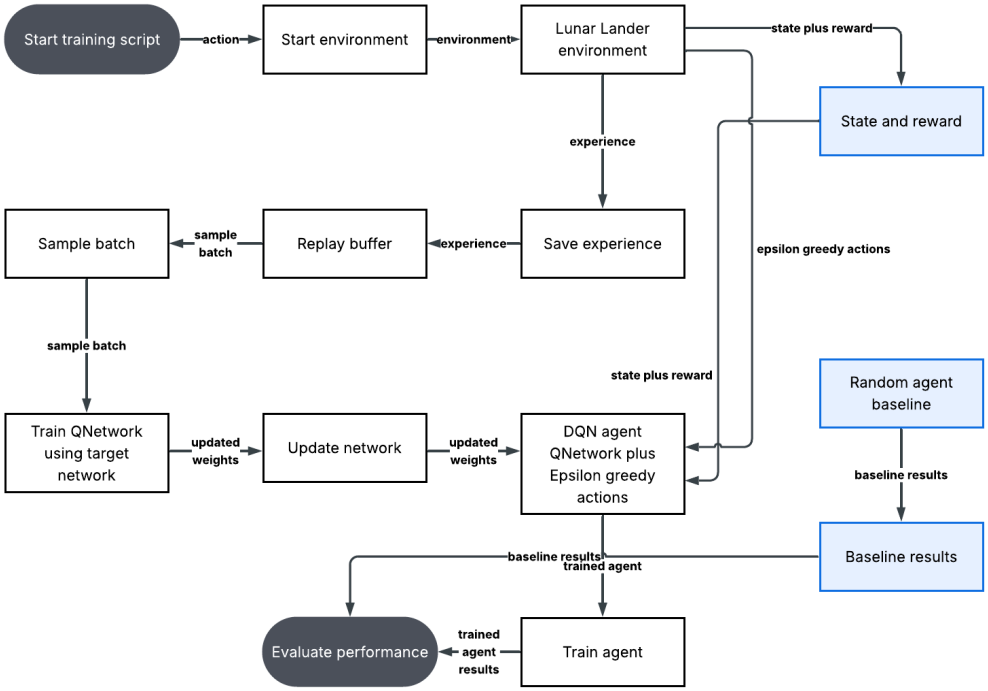
22 Tech Stack

- Python 3.13
- PyTorch for the Q-network and training
- Gymnasium and Box2D for LunarLander-v3
- NumPy for state and reward data
- Git and GitHub for version control
- Local files for checkpoints and training logs
- No external API; data comes from the simulation

2.3 Chunked Build Plan

- Set up the repository, Python environment, and dependencies — completed
- Test the observation space, action space, and physics — completed
- Create a reproducible random-agent baseline — completed
- Implement the PyTorch Q-network
- Add the replay buffer and epsilon-greedy action selection
- Add the target network and DQN update step
- Build the training loop with checkpoints and logs
- Evaluate against the random baseline and update the README and demo

2.4 System Diagram



3.1 GitHub Repository

https://github.com/notananotheralt/lunar-lander-dq